Automatic Image Analysis

Residual Networks

c.f. Prince, 2024

Solving Problems of Deep Nets

- Parameter initialization
- Residual connections
- Batch normalization

Initialization of Network Parameters

- How to initialize parameters before starting training?

$$\begin{aligned}\mathbf{f}_k &= \boldsymbol{\beta}_k + \boldsymbol{\Omega}_k \mathbf{h}_k \\ &= \boldsymbol{\beta}_k + \boldsymbol{\Omega}_k \mathrm{a}[\mathbf{f}_{k-1}]\end{aligned}$$

- Imagine, we initialize **biases to zero** and weights according to a **normal distribution with mean zero and variance** σ^2.

1)If the **variance is very small**, the activations of the next layer will likely have a **much smaller magnitude** than the input.

2)If the **variance is very large**, the activations of the next layer will likely have a **much larger magnitude** than the input.

- The same applies to the backwards pass
  →**vanishing gradient, exploding gradient** problem.

Initialization of Network Parameters

- Investigating the **expectation** (expected values) of the next layers activations and
- considering that half of the pre-activations will be below zero (zero mean normal distribution)
  and will therefore be **clipped by the ReLU function**
- as a consequence of the **dot product**,
  the following equation can be proven

$$\sigma_{f'}^2 = \sigma_\Omega^2 \sum_{j=1}^{D_h} \frac{\sigma_f^2}{2} = \frac{1}{2} D_h \sigma_\Omega^2 \sigma_f^2$$

- So if we want the variance of the next layer to be the same as the input layer, we should set

$$\sigma_\Omega^2 = \frac{2}{D_h}$$

with D_h being the dimension of the input layer.

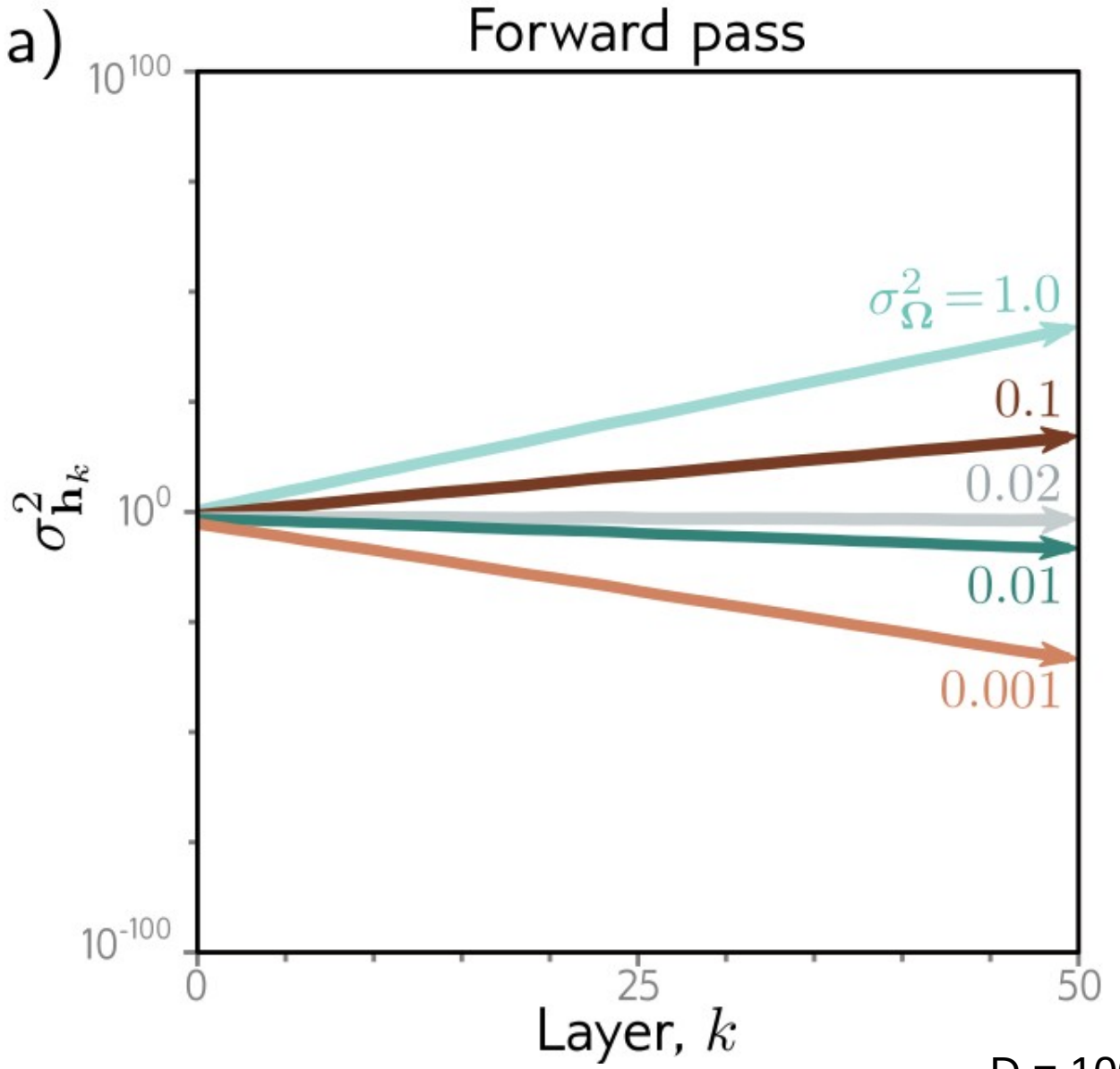
Initialization of Network Parameters

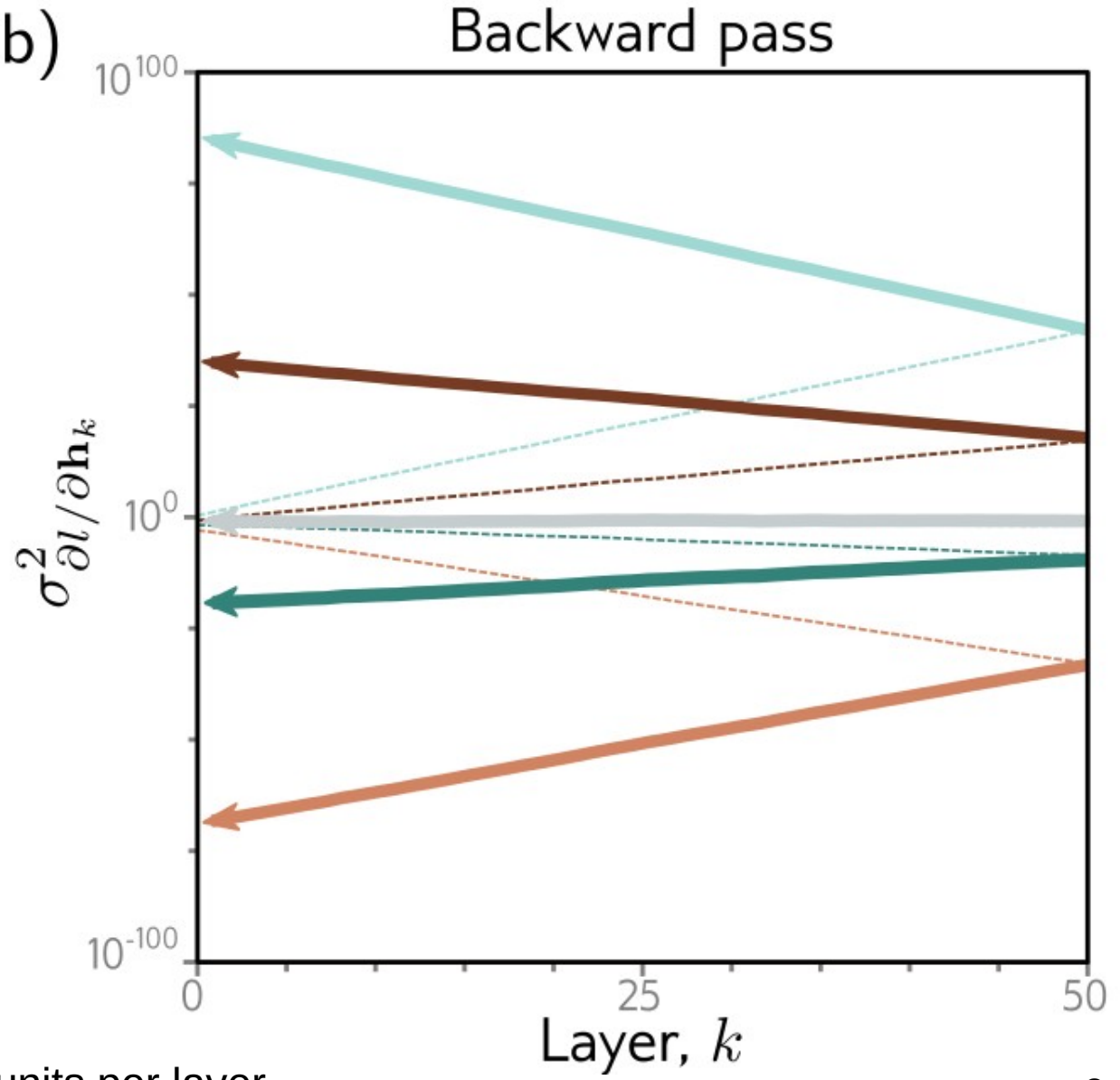
- As the same argumentation is valid for the backwards pass, we should consider both, and therefore set

$$\sigma_\Omega^2 = \frac{4}{D_h + D_{h'}}$$

- This is known as He or Kaiming initialization:
  HE Kaiming: Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification (2015).

Initialization of Network Parameters





D = 100 units per layer

He initialization (assumes ReLU)

- Forward pass: want the variance of hidden unit activations in layer k+1 to be the same as variance of activations in layer k:

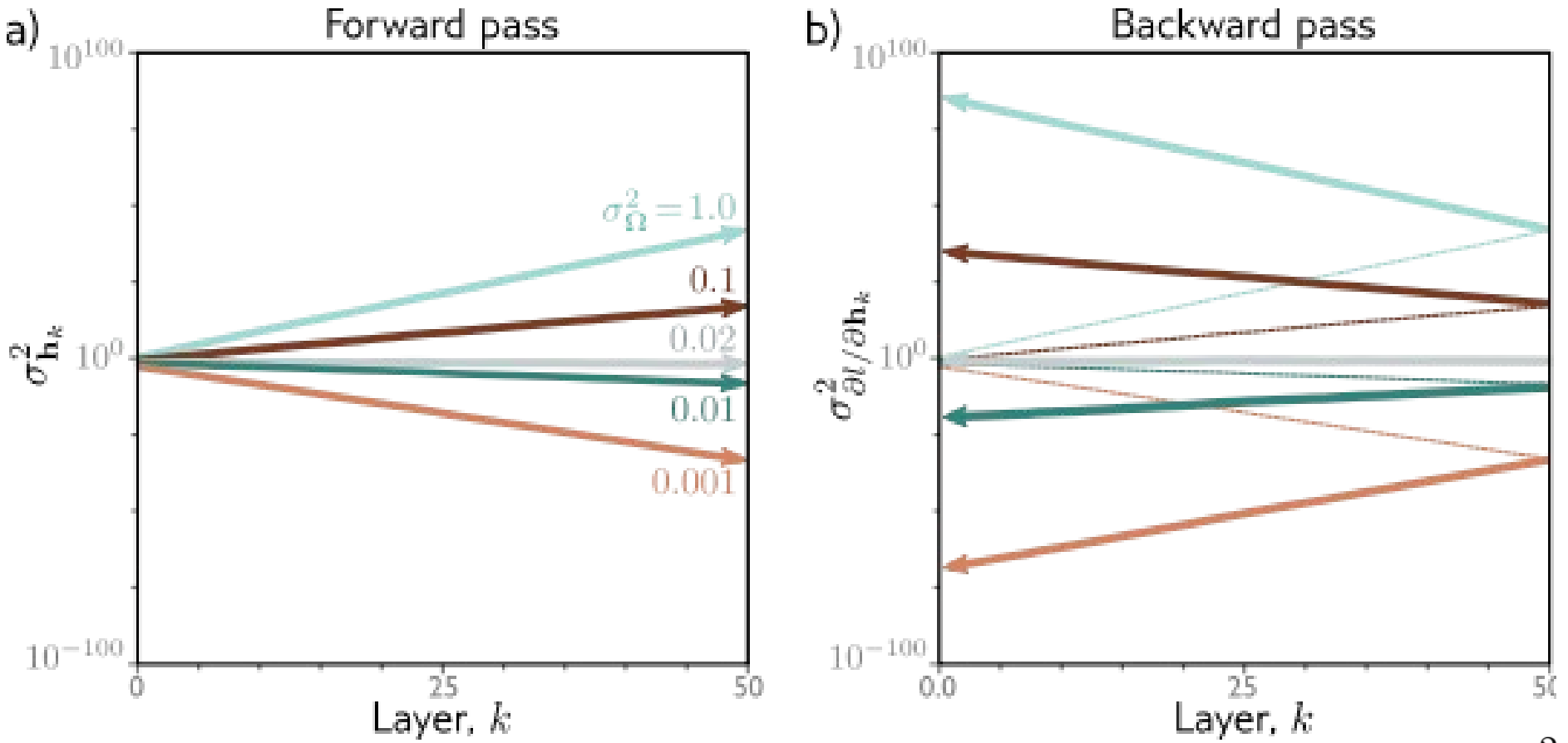
$$\sigma_{\Omega}^{2} = \frac{2}{D_h}$$

← Number of units at layer k

- Backward pass: want the variance of gradients at layer k to be the same as variance of gradient in layer k+1:

$$\sigma_{\Omega}^{2} = \frac{2}{D_{h'}}$$

← Number of units at layer k+1



Exploding gradients

Vanishing gradients

$$\sigma^2_\Omega = \frac{2}{D_h} = \frac{2}{100} = 0.02$$

Figure 7.4 Weight initialization. Consider a deep network with 50 hidden layers and $D_h = 100$ hidden units per layer. The network has a 100 dimensional input $\mathbf{x}$ initialized with values from a standard normal distribution, a single output fixed at $y = 0$, and a least squares loss function. The bias vectors $\boldsymbol{\beta}_k$ are initialized to zero and the weight matrices $\boldsymbol{\Omega}_k$ are initialized with a normal distribution with mean zero and five different variances $\sigma^2_\Omega \in \{0.001, 0.01, 0.02, 0.1, 1.0\}$. a)

Solving Problems of Deep Nets

- Parameter initialization
- Residual connections
- Batch normalization

Are Deeper Networks Better?

- Networks process data sequentially.
- Every layer receives previous layer's output as input:

$$\begin{aligned}\mathbf{h}_1 &= \mathbf{f}_1[\mathbf{x}, \boldsymbol{\phi}_1] \\ \mathbf{h}_2 &= \mathbf{f}_2[\mathbf{h}_1, \boldsymbol{\phi}_2] \\ \mathbf{h}_3 &= \mathbf{f}_3[\mathbf{h}_2, \boldsymbol{\phi}_3] \\ \mathbf{y} &= \mathbf{f}_4[\mathbf{h}_3, \boldsymbol{\phi}_4]\end{aligned}$$

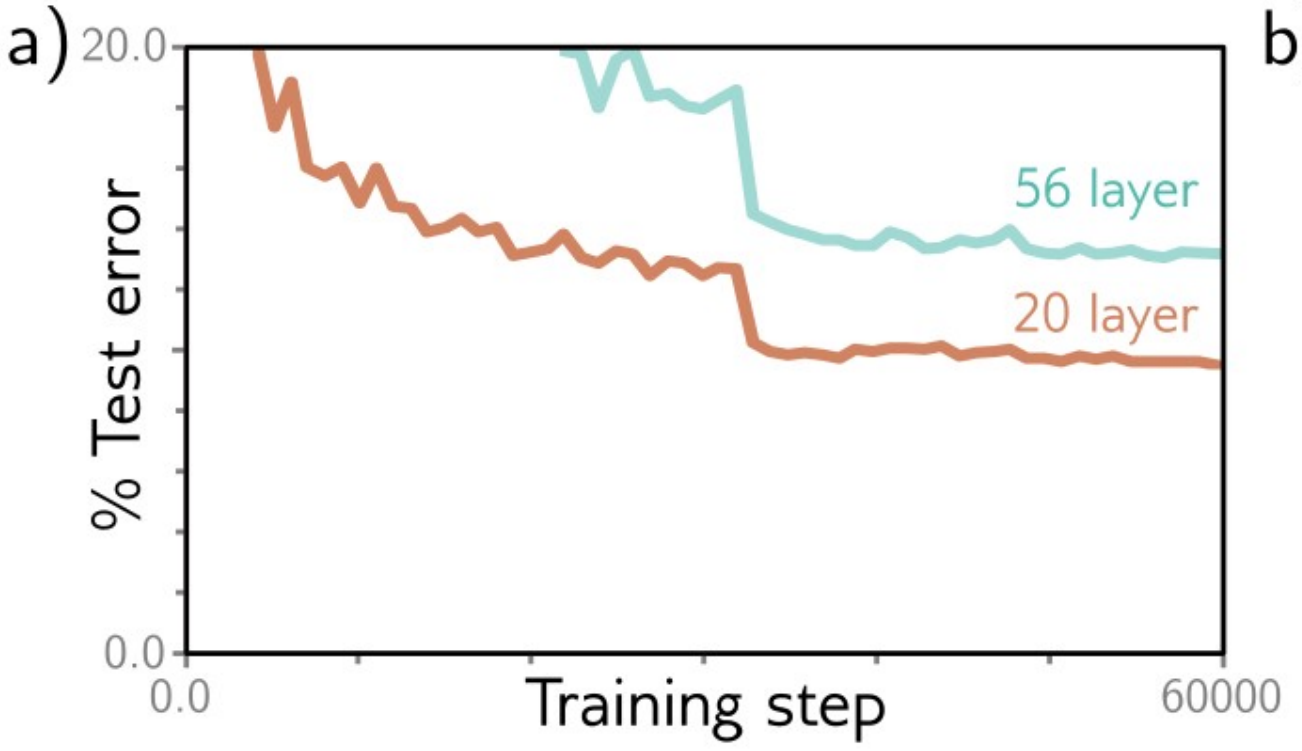
- This can be formulated as a series of nested functions

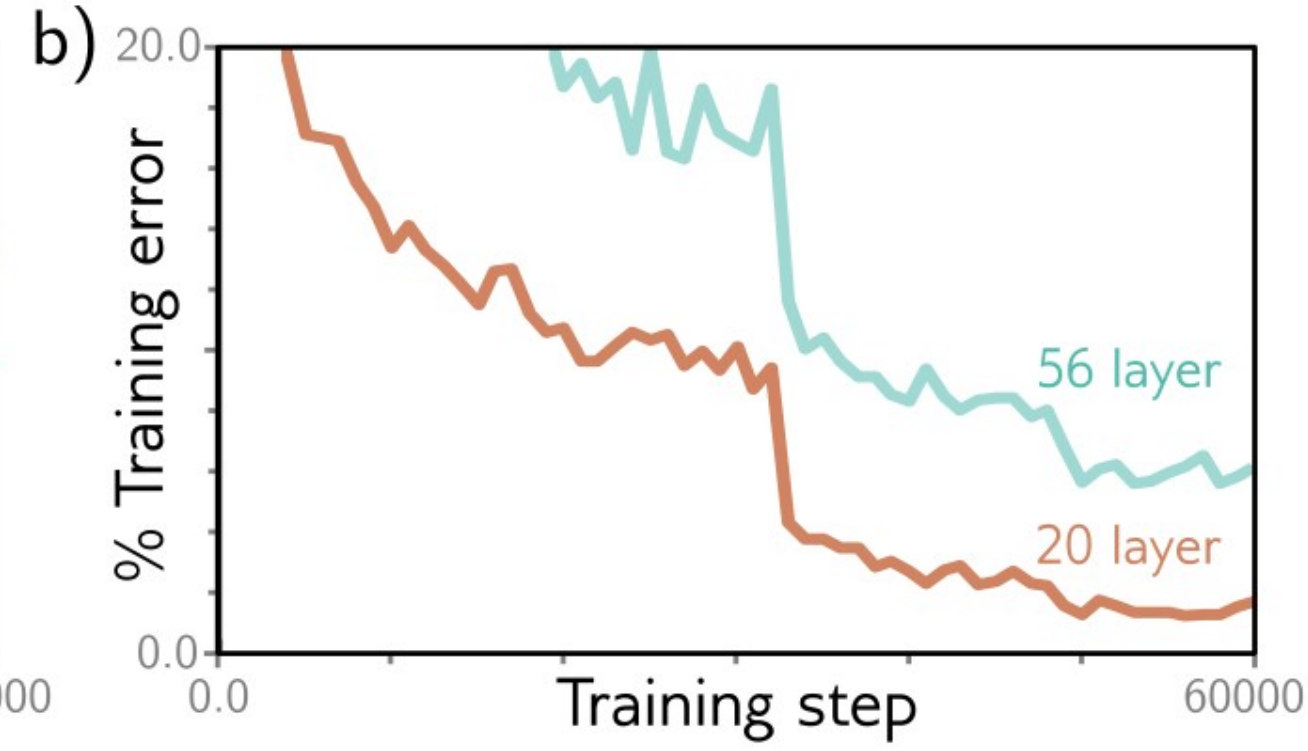
$$\mathbf{y} = \mathbf{f}_4\Big[\mathbf{f}_3\Big[\mathbf{f}_2\big[\mathbf{f}_1[\mathbf{x}, \boldsymbol{\phi}_1], \boldsymbol{\phi}_2\big], \boldsymbol{\phi}_3\Big], \boldsymbol{\phi}_4\Big]$$

- Adding more layers improves performance:
  - Example: VGG with 18 layers outperforms AlexNet with 8 layers.

Decrease in Performance When Adding More Convolutional Layers

- Not only for test data, but already for training data, error increases for more layers.
- Example for CIFAR-10 dataset:

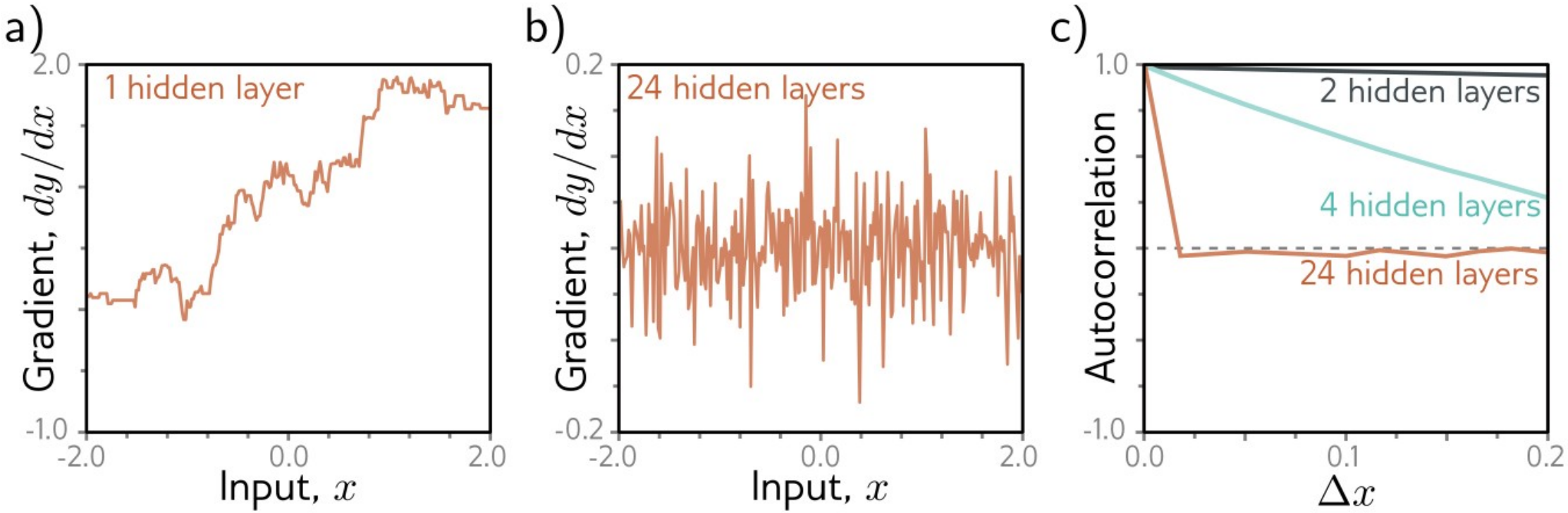




- He et al. (2016)

Shattered Gradients I

- Optimization algorithms approximate infinitesimal step size by numeric differentiation with finite step size.
- For deeper nets, gradients $\partial y / \partial x$ change quickly and unpredictably for small changes Δx



Shattered Gradients II

- **Shattered gradients** presumably arise, because
  - changes in early layers modify the output in an **increasingly complex** way
  - **the deeper the network** is.
- Derivative of the output y w.r.t. the first layer f_1:

$$\frac{\partial \mathbf{y}}{\partial \mathbf{f}_1} = \frac{\partial \mathbf{f}_4}{\partial \mathbf{f}_3}\frac{\partial \mathbf{f}_3}{\partial \mathbf{f}_2}\frac{\partial \mathbf{f}_2}{\partial \mathbf{f}_1}$$

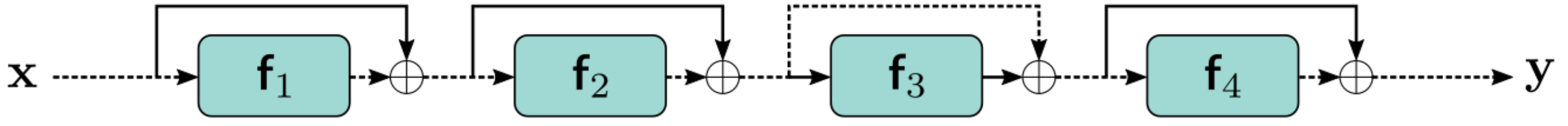
- When parameters that determine f_1 change,
  all of the derivatives in the sequence change,
  as they are computed from f_1.

Remedy: Residual Connections

- Residual (or **skip) connections** are branches in the computational path, whereby the **input of each network layer is added to the output**:

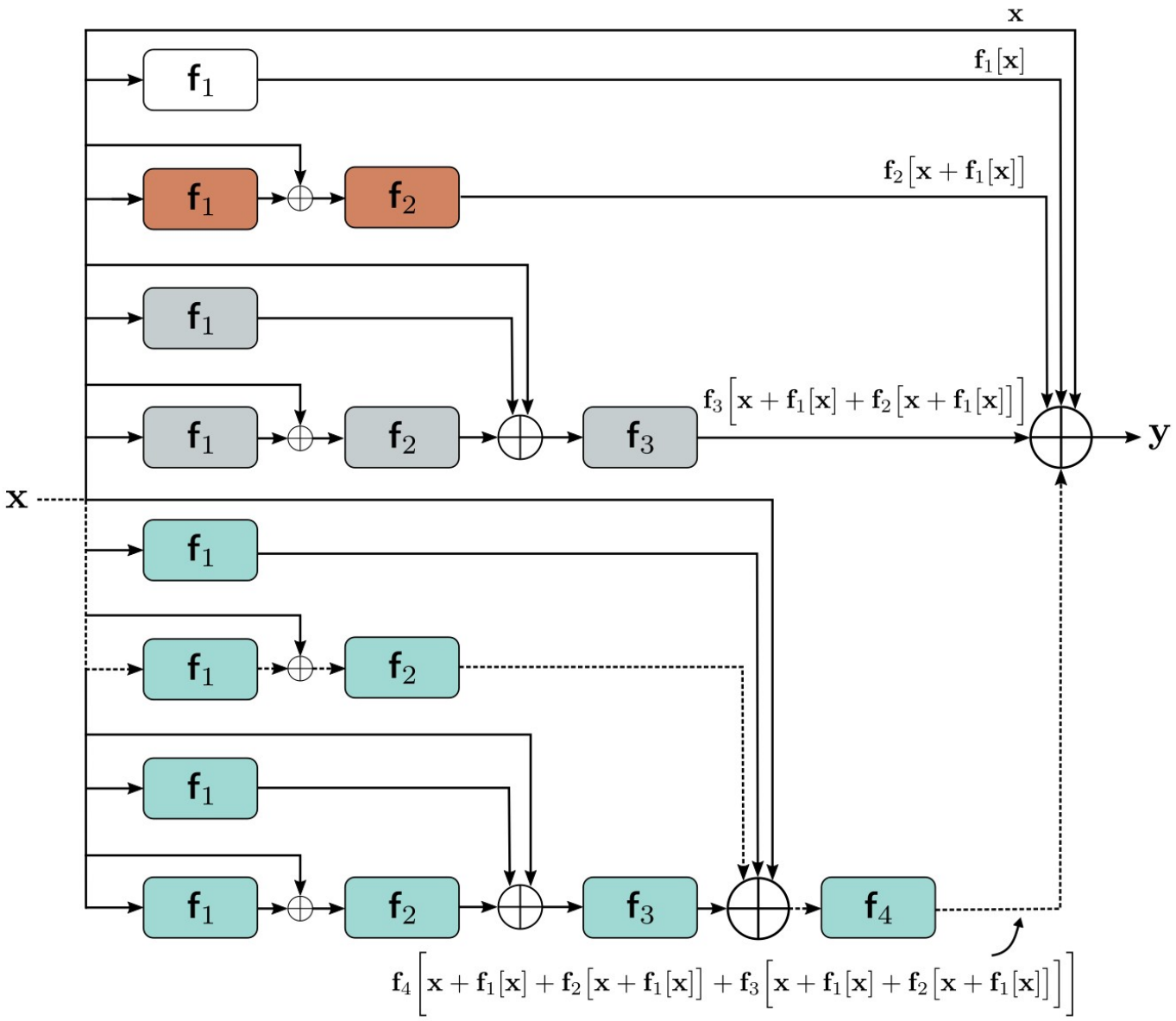
$$
\begin{aligned}
\mathbf{h}_1 &= \mathbf{x} + \mathbf{f}_1[\mathbf{x}, \boldsymbol{\phi}_1] \\
\mathbf{h}_2 &= \mathbf{h}_1 + \mathbf{f}_2[\mathbf{h}_1, \boldsymbol{\phi}_2] \\
\mathbf{h}_3 &= \mathbf{h}_2 + \mathbf{f}_3[\mathbf{h}_2, \boldsymbol{\phi}_3] \\
\mathbf{y} &= \mathbf{h}_3 + \mathbf{f}_4[\mathbf{h}_3, \boldsymbol{\phi}_4]
\end{aligned}
$$

$$
\begin{aligned}
\mathbf{y} = \mathbf{x} &+ \mathbf{f}_1[\mathbf{x}] \\
&+ \mathbf{f}_2\left[\mathbf{x} + \mathbf{f}_1[\mathbf{x}]\right] \\
&+ \mathbf{f}_3\left[\mathbf{x} + \mathbf{f}_1[\mathbf{x}] + \mathbf{f}_2\left[\mathbf{x} + \mathbf{f}_1[\mathbf{x}]\right]\right] \\
&+ \mathbf{f}_4\left[\mathbf{x} + \mathbf{f}_1[\mathbf{x}] + \mathbf{f}_2\left[\mathbf{x} + \mathbf{f}_1[\mathbf{x}]\right] + \mathbf{f}_3\left[\mathbf{x} + \mathbf{f}_1[\mathbf{x}] + \mathbf{f}_2\left[\mathbf{x} + \mathbf{f}_1[\mathbf{x}]\right]\right]\right]
\end{aligned}
$$



Residual Blocks

- Unraveling the network equations (2nd equation):

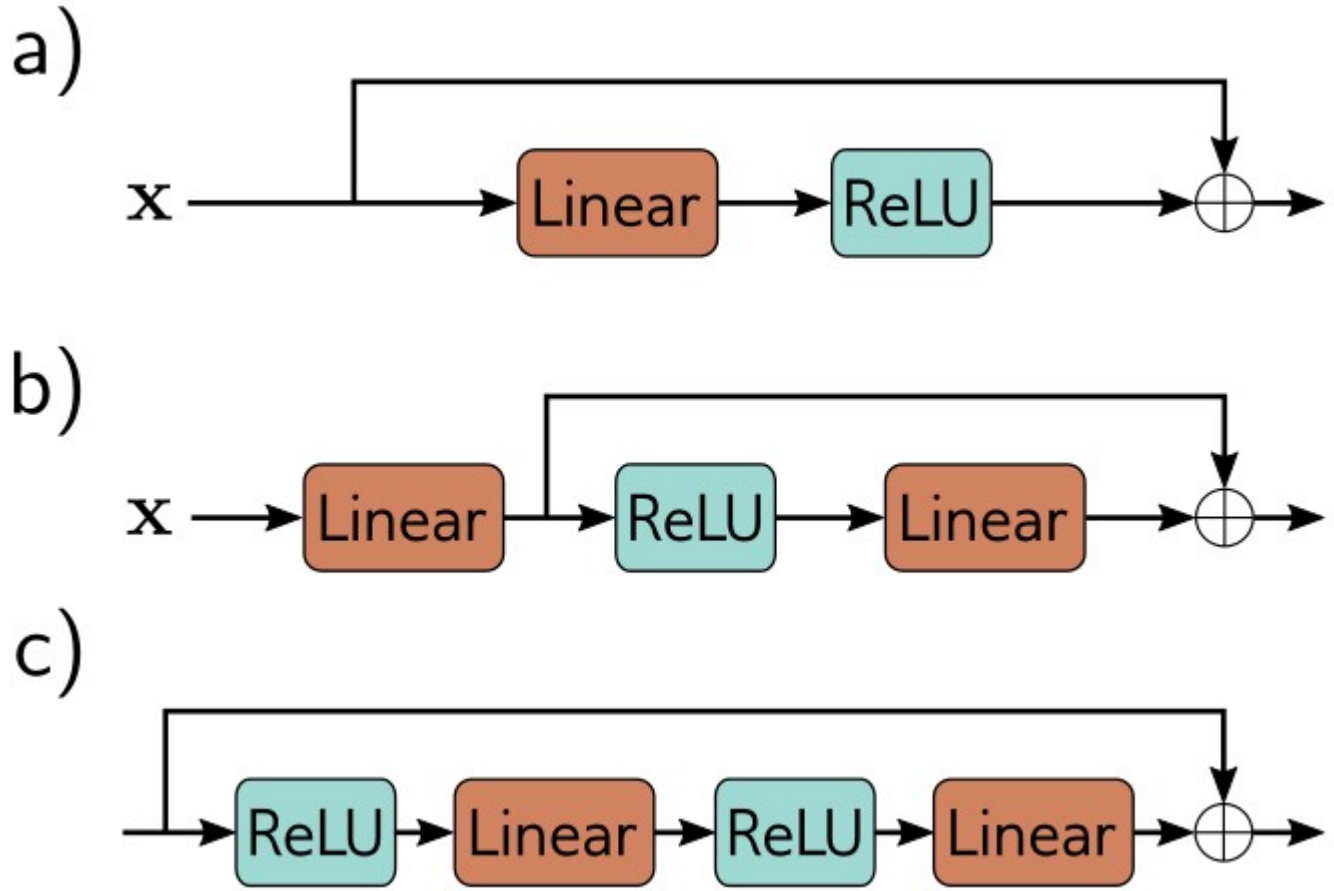


Order of Operations in Residual Blocks

a) If ReLU is at the end, each residual block can only increase the input values. So ReLU is done at the beginning.

b) If ReLU is at the beginning, the residual block does nothing if the input is negative. So the network is started with a linear transformation.

c) It is common for a residual block to contain several network layers.



Deeper Networks with Residual Connections

- are more stable than without residual connections:
- Adding residual connections **roughly doubles the depth** of a network that can be practically trained **before performance degrades**.
- However, **residual networks still suffer from unstable** forward propagation and exploding gradients.
- Counter measure:
  - He initialization
  - **Multiplying** the combined output of each residual block **by 1/sqrt(2)**
    - Due to the addition of the input to the output of a residual block variance doubles. This is compensated for by multiplying by 1/sqrt(2).
- More common: **batch normalization**

Solving Problems of Deep Nets

- Parameter initialization
- Residual connections
- Batch normalization

Batch Normalization

- Rescales each activation **across the batch**'s training examples.
- First, empirical mean *m* and standard deviation *s* are computed:

$$m_h = \frac{1}{|\mathcal{B}|}\sum_{i\in\mathcal{B}} h_i$$

$$s_h = \sqrt{\frac{1}{|\mathcal{B}|}\sum_{i\in\mathcal{B}}(h_i - m_h)^2}$$

- Then standardize to **mean 0** and **variance 1**:

$$h_i \leftarrow \frac{h_i - m_h}{s_h + \epsilon} \qquad \forall i \in \mathcal{B}$$

- **Learn** mean δ and standard deviation γ **during training**:

$$h_i \leftarrow \gamma h_i + \delta \qquad \forall i \in \mathcal{B}$$

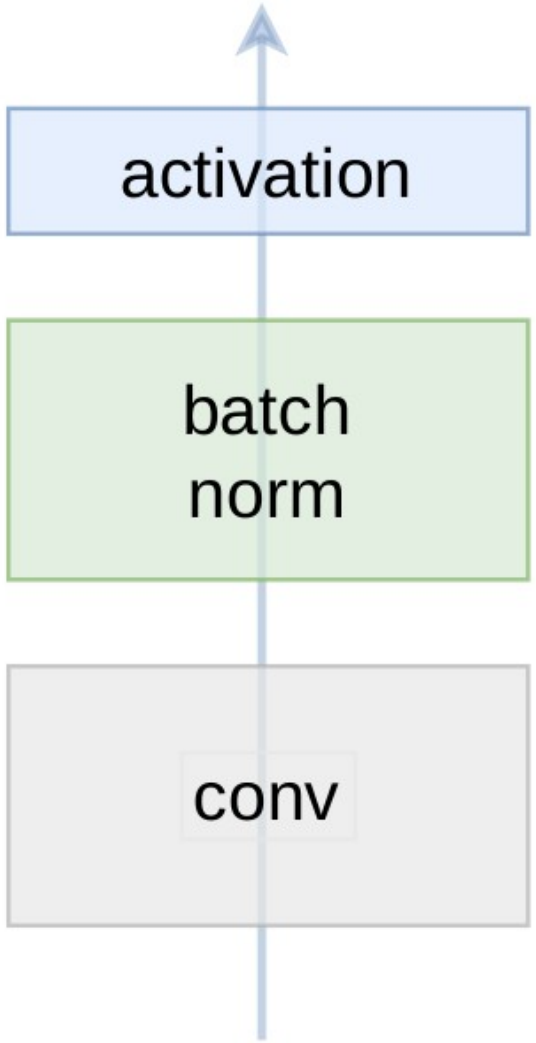
Batch Normalization

- Is applied independently to each hidden unit.
- So in a **fully-connected network** with K layers and D hidden units KD learned offsets and scales are computed.
- In a **convolutional network** normalizing statistics is not only computed across the batch, but also over spatial position.
- So KC offsets and KC scales are computed,
  where C is the number of channels.

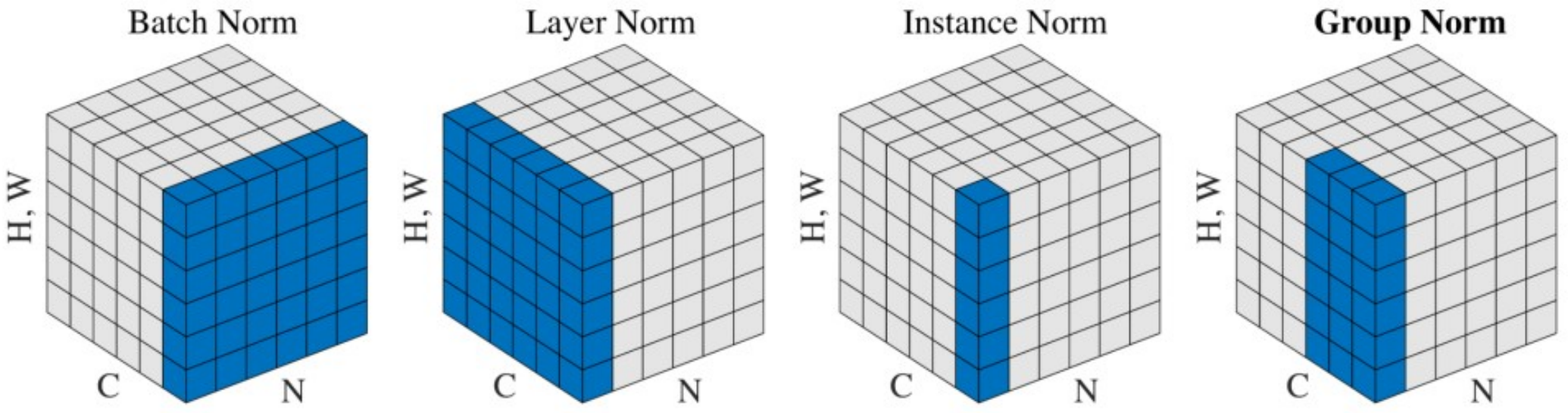
- Summary:
  Batch **normalization of activations** makes the network **invariant to rescaling weights and biases** -
  as it compensates for it.

Batch Normalization

- Usually inserted before the non-linearity



Alternative Forms of Normalization

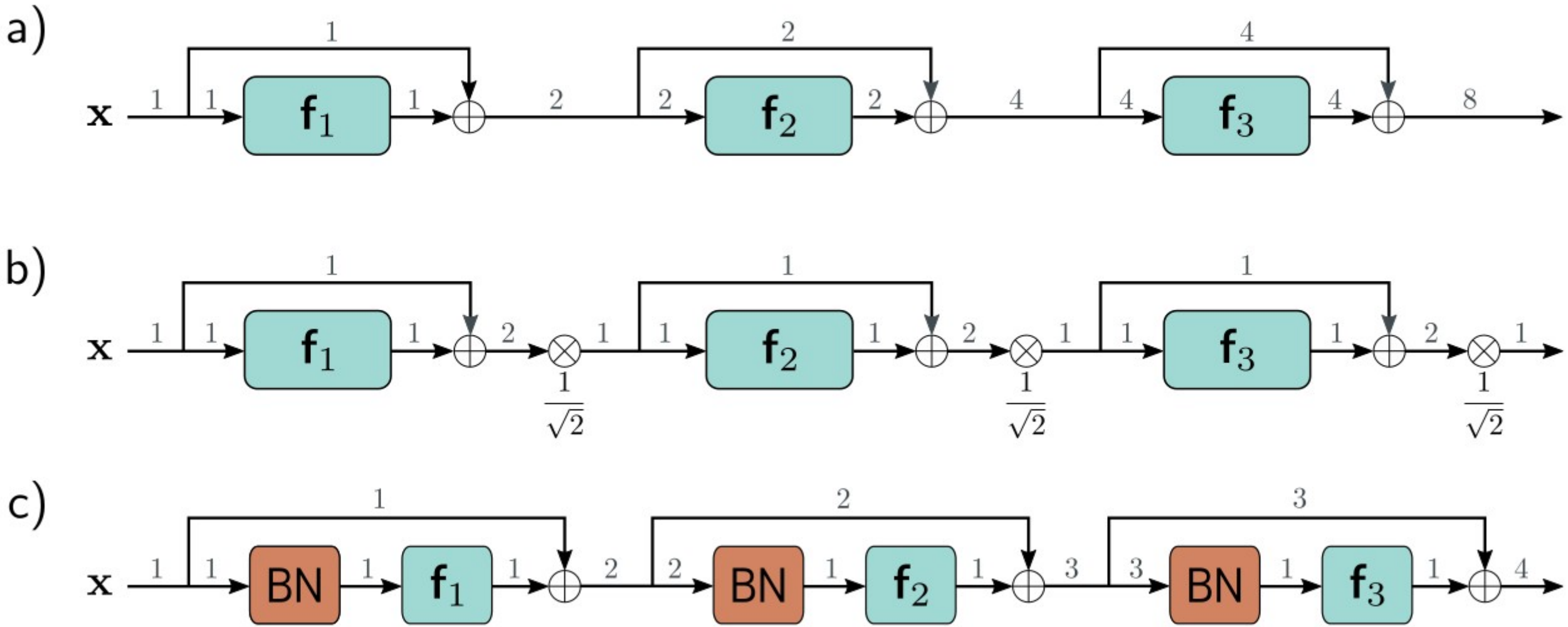


- Layer Normalization, Ba et al, 2016
- Improved Texture Networks: Maximizing Quality and Diversity in Feed-forward Stylization and Texture Synthesi, Ulyanov et al, 2017
- Group Normalization, Wu & He, Group normalization, 2018
- Many more including combinations of these and weight normalization
- Image from Group Normalization, Wu & He, Group normalization, 2018

Costs and Benefits of Batch Normalization

- Stable forward propagation
  - With **initialization mean = 0, variance = 1**, each output activation will have unit variance.
  - In regular networks (no residual connections), this ensures that the variance is stable during forward propagation
  - In a residual network, variance increases as the input is added to the output.
  - Then later layers are having a smaller impact.
  - I.e. the network is **effectively less deep**.
  - While training proceeds, **mean** and **variance** can be increased. Then **“the network”** (training, backpropagation) **controls its own effective depth**.

Variance in Residual Networks



Costs and Benefits of Batch Normalization

- Higher learning rates:
  - Batch normalization makes loss surface and its gradients change more smoothly.
  - This allows higher learning rates.
- Regularization:
  - Batch normalization injects noise, because the normalization depends on the batch’s statistics.
  - Adding noise can improve generalization.

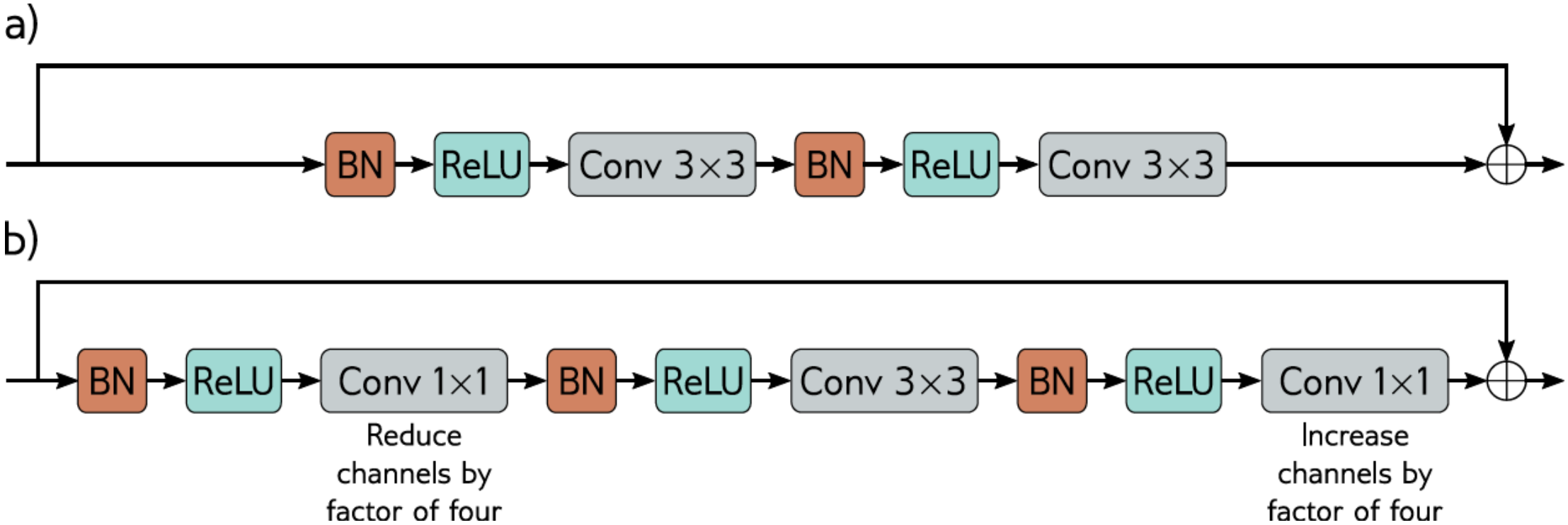
Common Residual Architectures

- ResNet
- DenseNet
- U-Nets
- Hourglass networks

Resnet Blocks

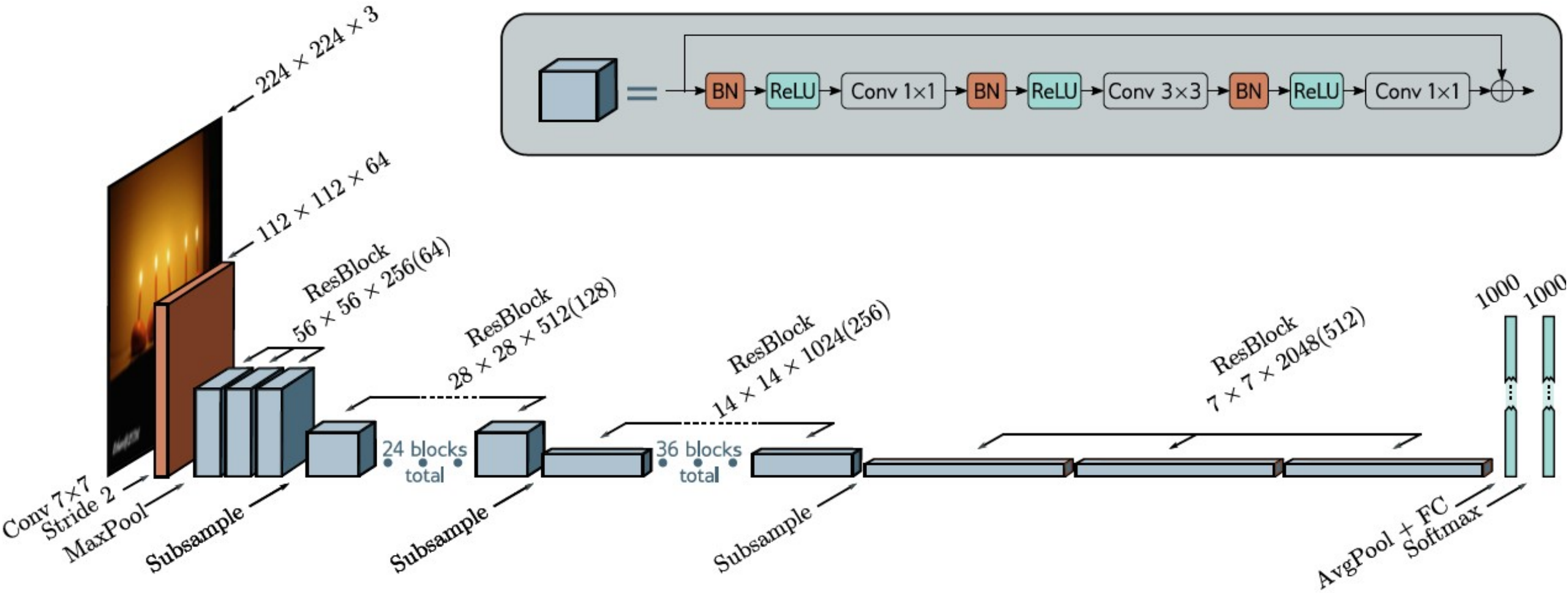
a) Standard block with **batch normalization** followed by an **activation function**, and a **convolutional layer**.

b) Bottleneck ResNet block first **reduces number of channels** with **1x1 convolution** and afterwards increases the number of layers again.



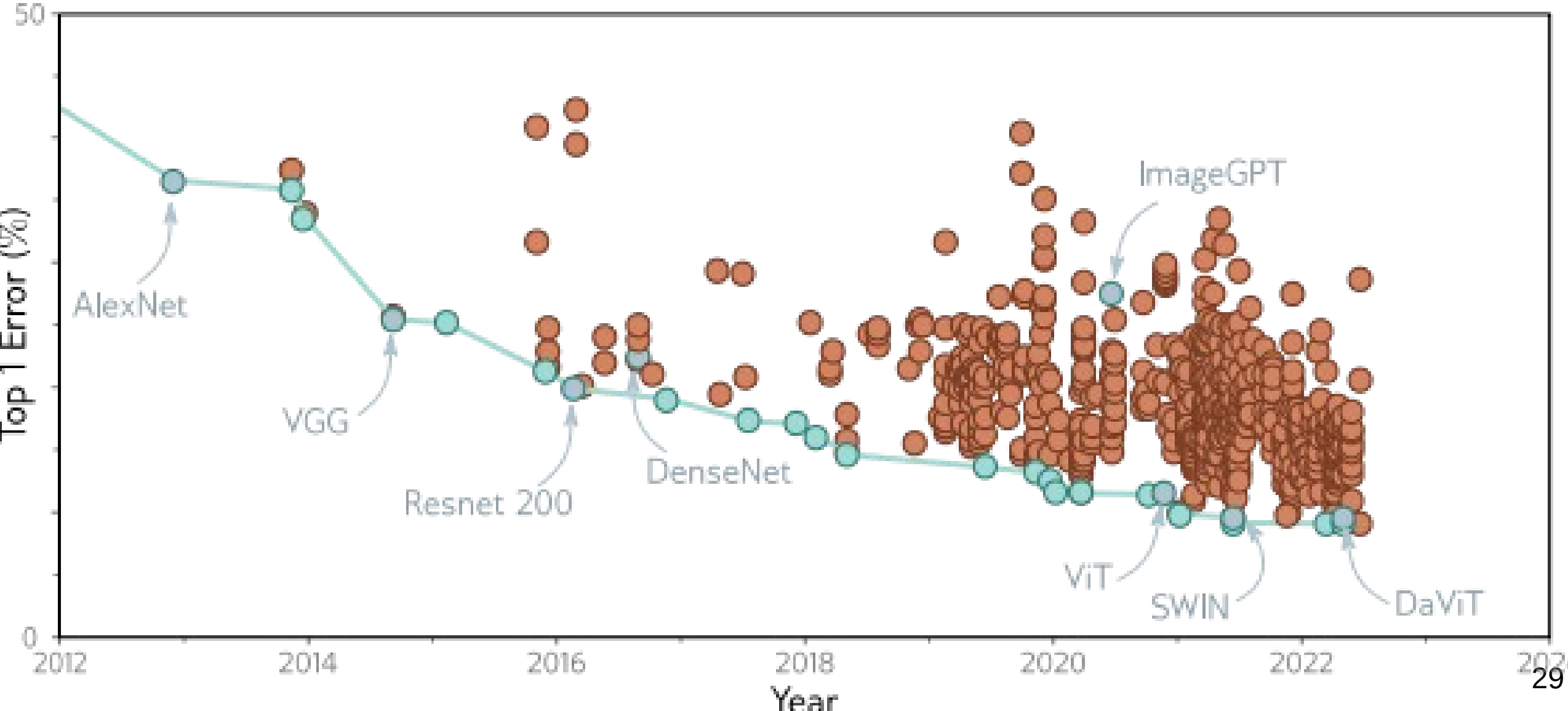
Resnet 200 (2016)

He et al. (2016)



- Number in brackets:
  number of channels after first 1x1 convolution.
- Error on ImageNet 4.8%; first net exceeding human performance

ImageNet History



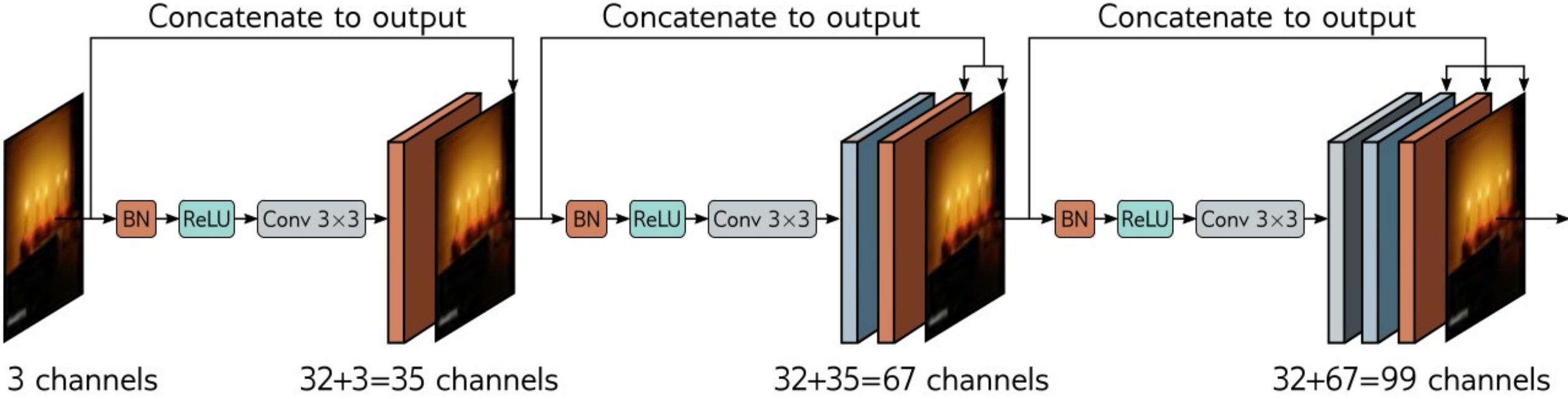
Common Residual Architectures

- ResNet
- DenseNet
- U-Nets
- Hourglass networks

DenseNet

Huang et al. (2017)

- Instead of **adding** input activations to the output, input activations can also be **concatenated** to the output.
- In DenseNet architecture input to the **next layer comprises concatenated output from all previous layers**.
- Then there is a **direct contribution from earlier layers** to the output – making the loss surface behave reasonably.



DenseNet

- In order not to let the number of layers become too large, **number of channels is reduced by applying a 1x1 convolution**.
- Concatenation across downsampling makes no sense since the representations have different sizes. Then the **chain of concatenations is broken**, and a **smaller representation starts a new chain**.
- DenseNet can perform better than ResNet having the same number of parameters.

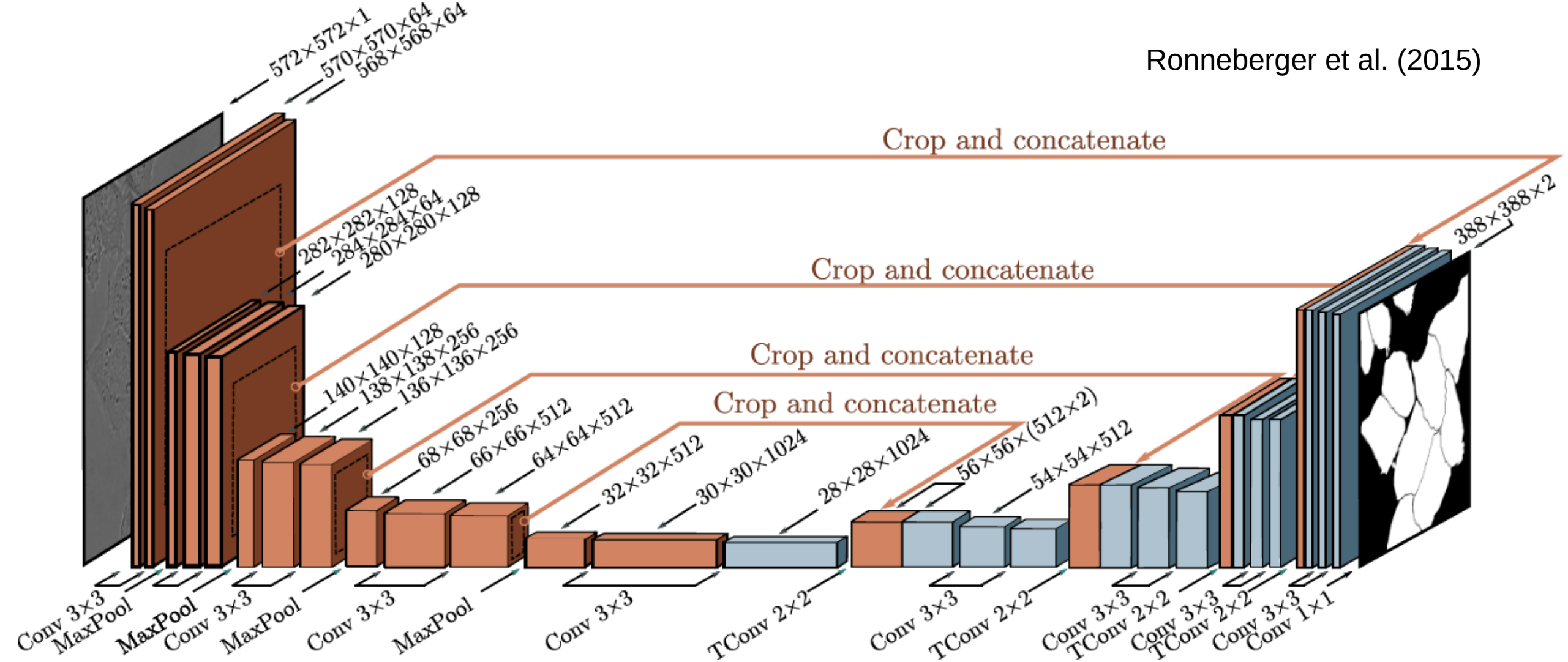
Common Residual Architectures

- ResNet
- DenseNet
- U-Net
- Hourglass networks

U-Net (2016)

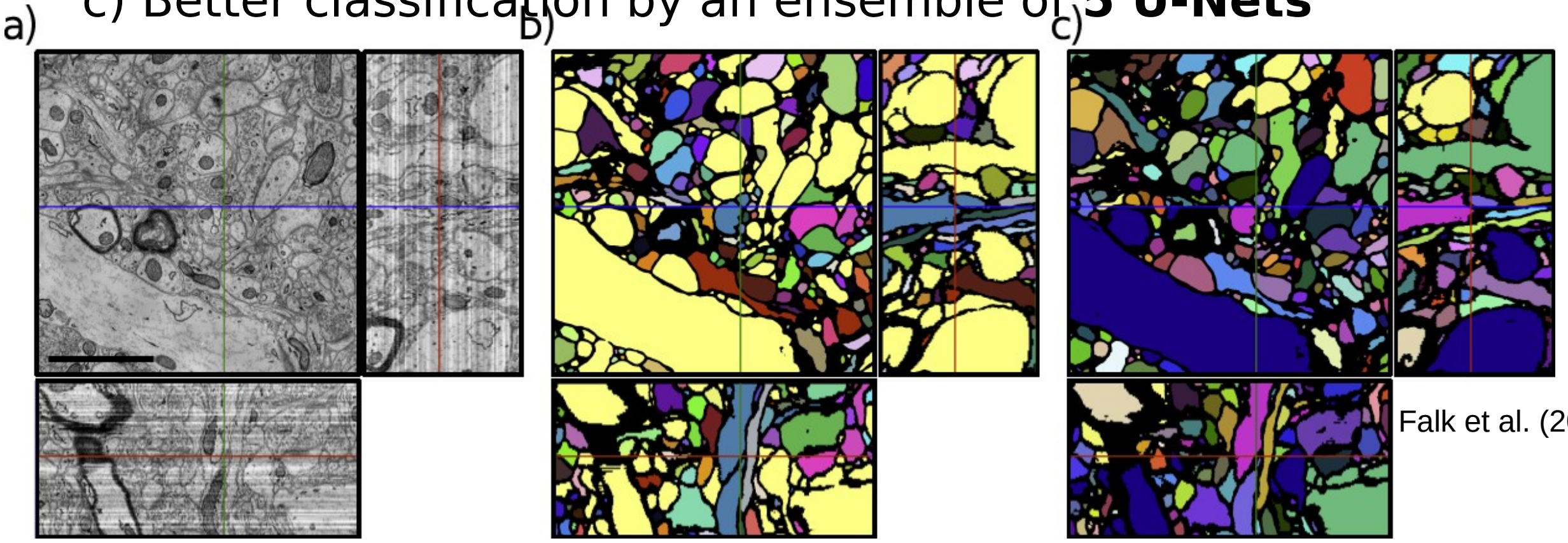
- With residual connections the encoder-decoder network does not need to “remember” the high-resolution detail.

Ronneberger et al. (2015)



U-Net Results

- U-Net was intended for segmenting **medical images**.
- Example: a) Three slices through a 3D volume of mouse cortex taken by a **scanning electron microscope**
  b) Classification of voxels by a **single U-Net**
  c) Better classification by an ensemble of **5 U-Nets**

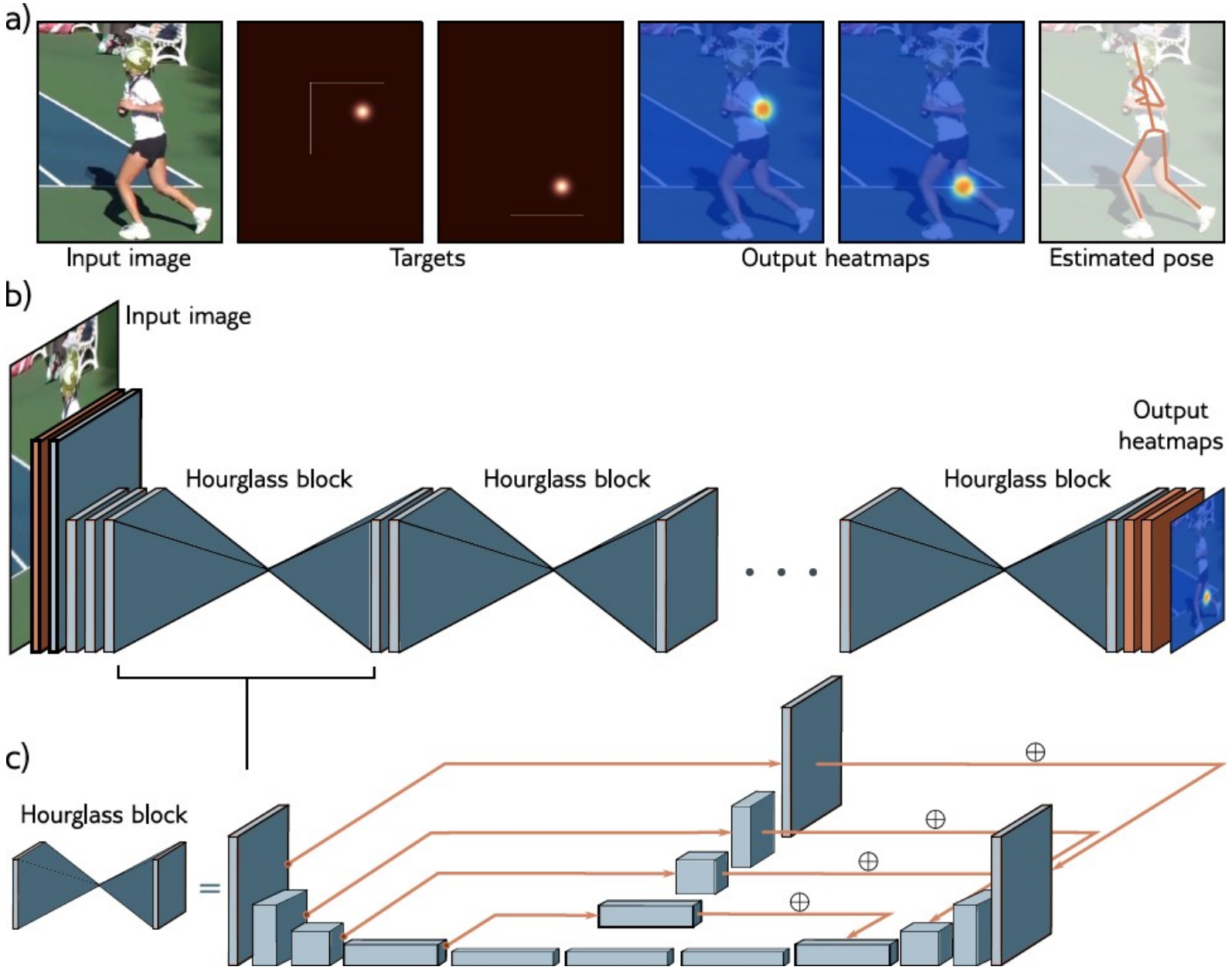


Falk et al. (2019)

Common Residual Architectures

- ResNet
- DenseNet
- U-Net
- Hourglass networks

Stacked Hourglass Networks (2016)



Newall et al. (2016)

Stacked Hourglass Networks (2016)

- **Adding** results **rather than concatenating**
- **Several U-Net-like blocks** sub-sequentially repeated alternating between considering the image at **local** and **global** levels
- Used for pose estimation:

  Producing one “heatmap” for each joint
  Estimated joint position is maximum of each “heatmap”
- Formulated as a **regression problem** (as opposed to a **classification problem**, we have seen so far)

Newall et al. (2016)

Why do Residual Nets perform so well?

- Loss surface